

LINEAR REGRESSION - EXPLAINED

Trong bài viết này, tôi sẽ nói về:

- Hồi quy tuyến tính là gì?
- Hàm chi phí
- Gradient descent

Linear Regression là gì?

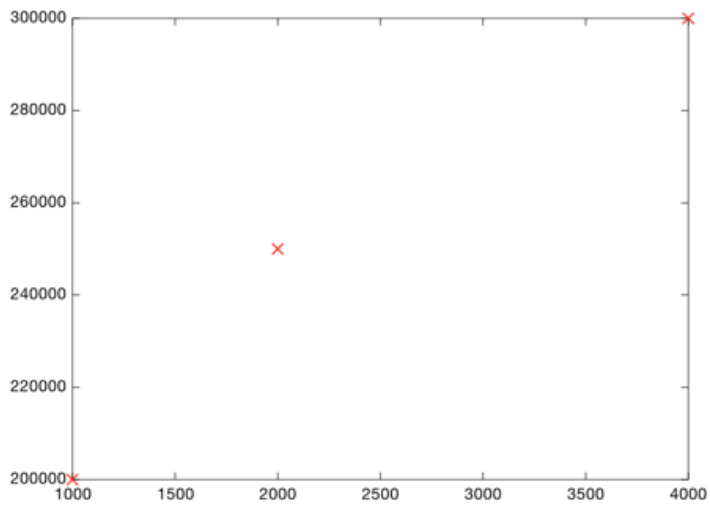
Giả sử bạn đang cân nhắc bán căn nhà của mình. Những ngôi nhà có kích thước khác xung quanh bạn đã được bán với các mức giá khác nhau.



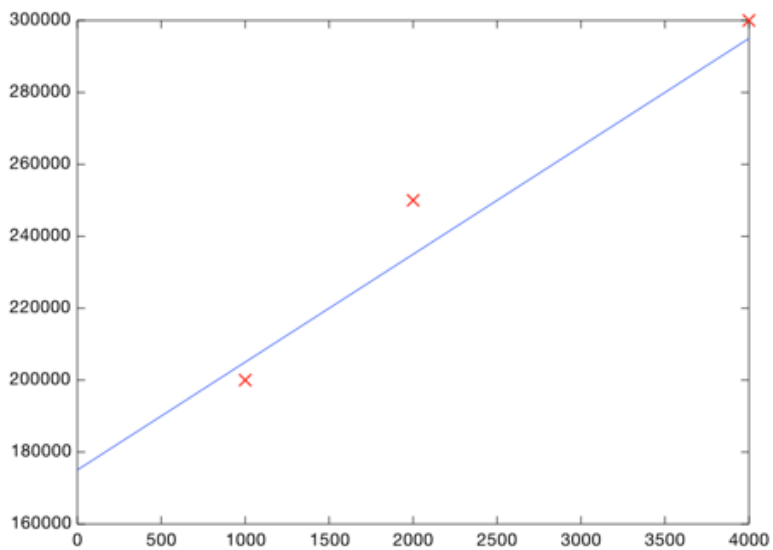
Nhà của bạn có diện tích 3000 feet vuông. Vậy bạn nên bán nó với giá bao nhiêu?

Bạn cần xem xét dữ liệu hiện có và dự đoán một mức giá cho ngôi nhà của mình. Quá trình này được gọi là **hồi quy tuyến tính (linear regression)**.

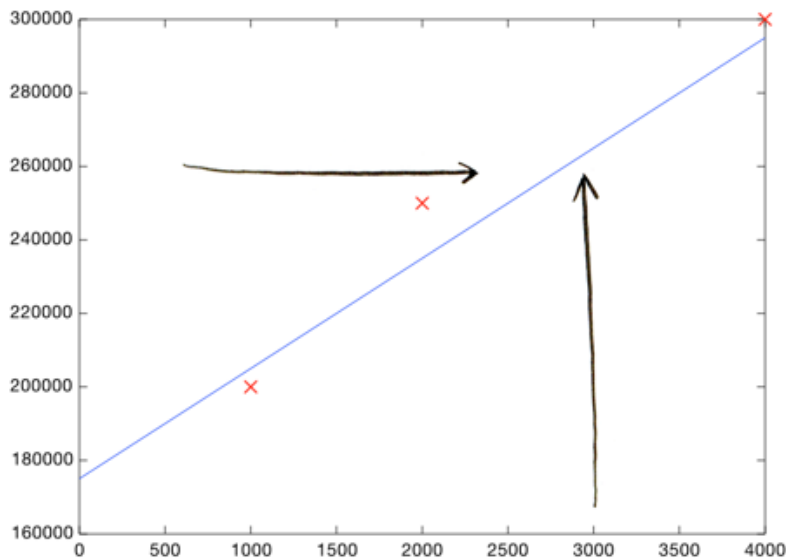
Dưới đây là 1 cách đơn giản để làm điều đó. Hãy xem dữ liệu mà bạn đang có:



Mỗi điểm trên đồ thị đại diện cho một ngôi nhà. Bây giờ bạn có thể nhìn nó bằng mắt và vẽ một đường thẳng gần nhất với tất cả các điểm dữ liệu này.

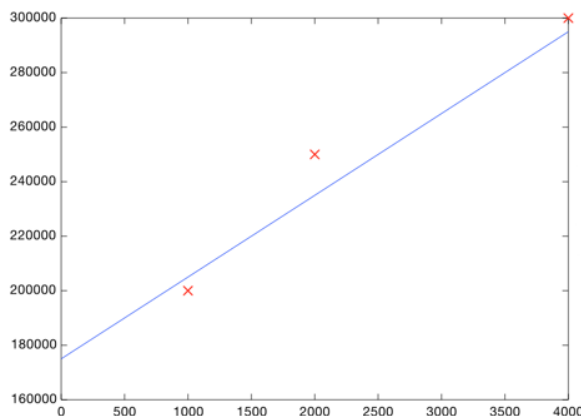


Sau đó, nhìn vào mức giá được xác định bởi đường thẳng tại vị trí có diện tích 3000 feet vuông:

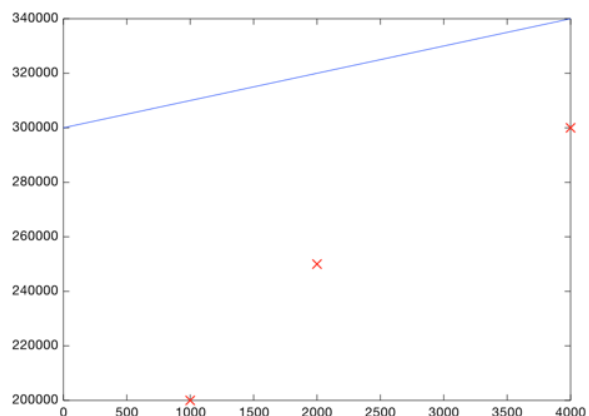


Tên ten! Ngôi nhà của bạn nên được bán với giá \$260,000.

Vậy là xong. Bạn chỉ cần **vẽ dữ liệu ra, ước lượng một đường thẳng** rồi dùng đường thẳng này **để đưa ra dự đoán**. Điều quan trọng là bạn phải đảm bảo rằng **đường thẳng đó khớp với dữ liệu một cách hợp lý**:



GOOD FIT

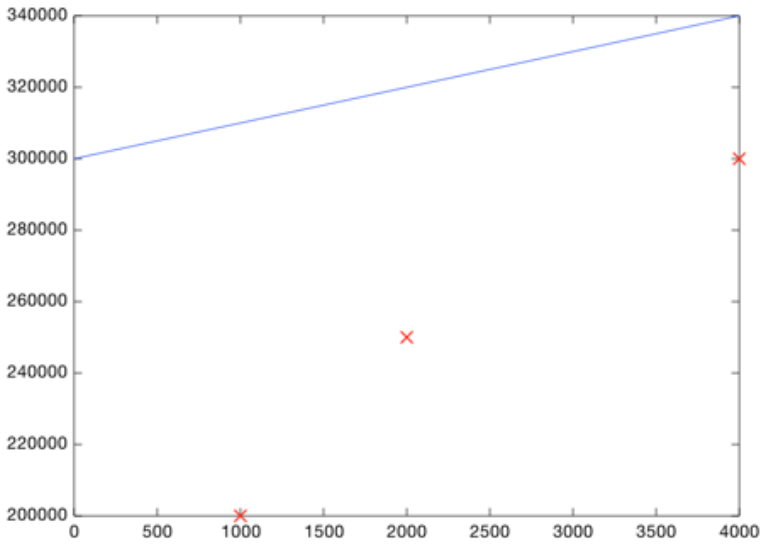


BAD FIT

Nhưng tất nhiên chúng ta không muốn dùng mắt để nhìn vào dữ liệu và vẽ 1 đường thẳng bất kỳ, chúng ta muốn máy tính phải phân tích và vẽ 1 đường thẳng “khớp” nhất với dữ liệu của chúng ta. Đó là lúc máy học xuất hiện.

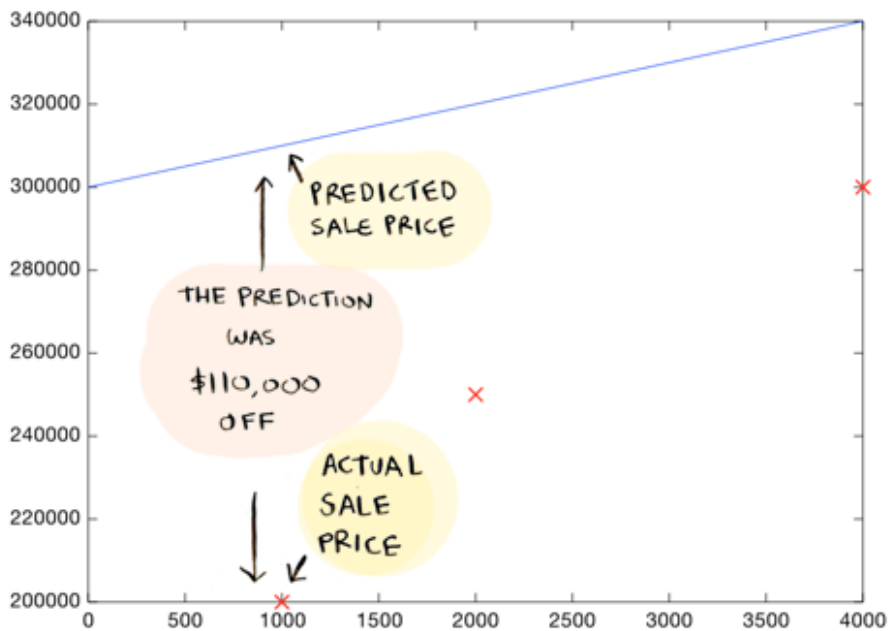
Vậy làm thế nào để máy tính xác định được đường thẳng nào là tốt?

Dưới đây là một ví dụ về một đường thẳng không tốt:



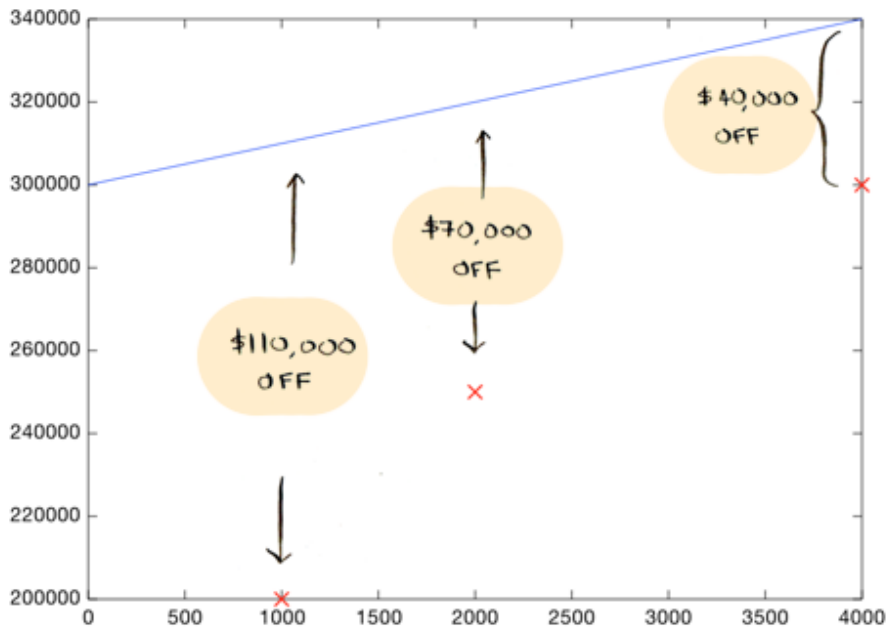
Đường thẳng này sai lệch quá nhiều.

Ví dụ, theo đường này thì một ngôi nhà rộng 1000 feet vuông nên được bán với giá \$310.000, trong khi thực tế, nó đã được bán với giá \$200.000.



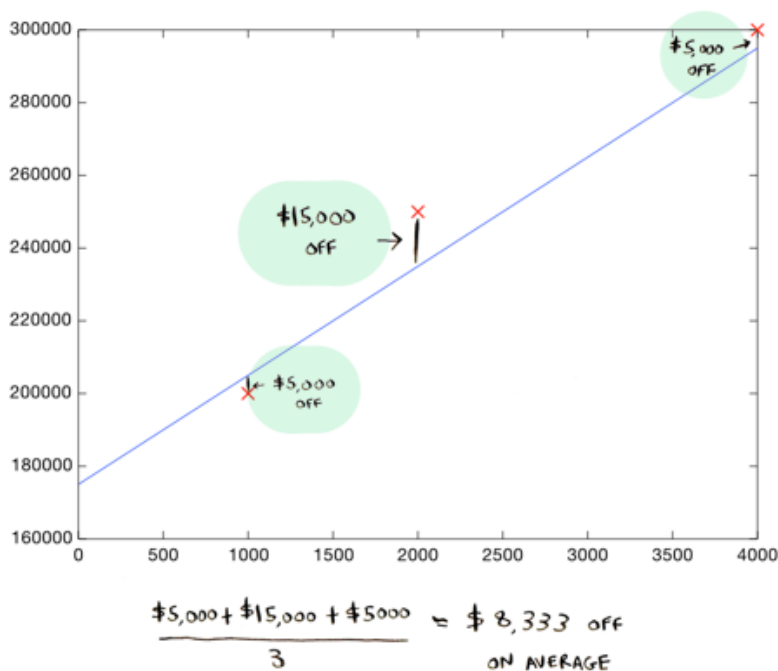
Có thể một người say rượu đã vẽ ra đường thẳng này... Nó lệch đến \$110.000 so với giá thực tế của ngôi nhà đó.

Và nó cũng lệch rất xa so với tất cả các giá trị khác nữa.



Trung bình, đường thẳng này lệch \$73,333 $(\$110,000 + \$70,000 + \$40,000)/3$.

Sau đây là một đường thẳng tốt hơn:



Đường thẳng này chỉ lệch trung bình \$8,333.. tốt hơn nhiều. Con số \$8,333 này được gọi là chi phí khi sử dụng đường thẳng này. **Chi phí** (cost) là mức độ sai lệch của đường thẳng so với dữ liệu thực tế.

Đường thẳng tốt nhất là đường thẳng có sai lệch nhỏ nhất so với dữ liệu thực tế.

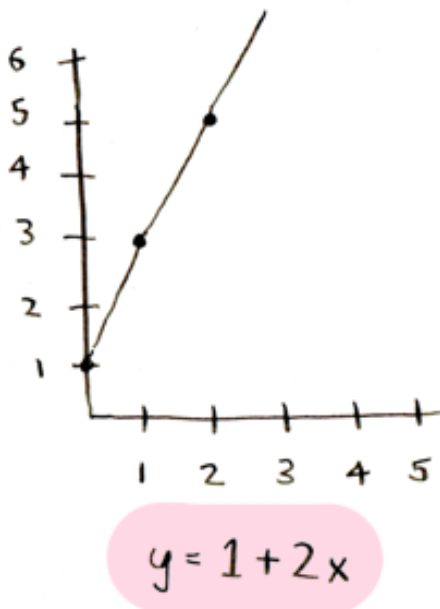
Và để tìm ra đường thẳng tốt nhất, chúng ta cần sử dụng một **hàm chi phí** (cost function).

The cost function

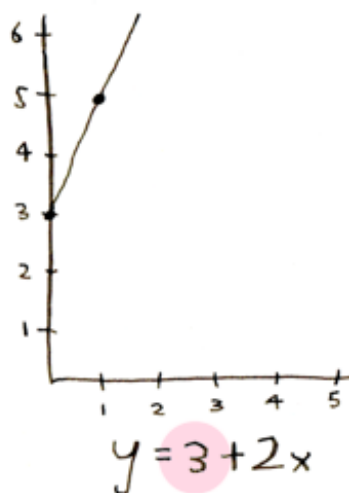
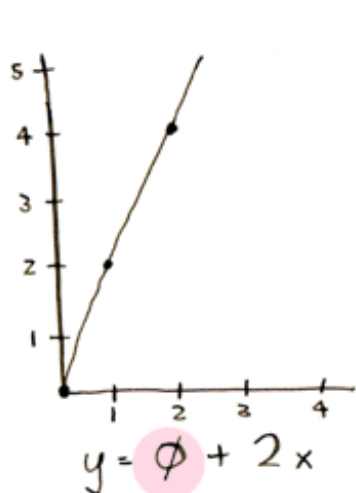


Trước tiên, hãy bắt đầu với một số kiến thức toán học cơ bản.

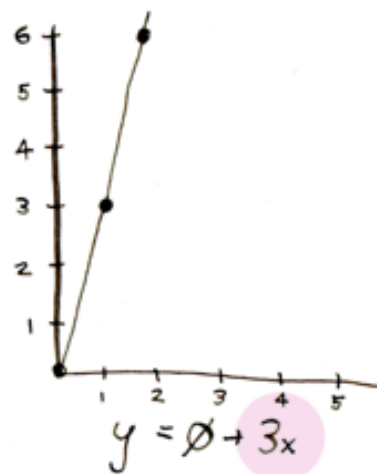
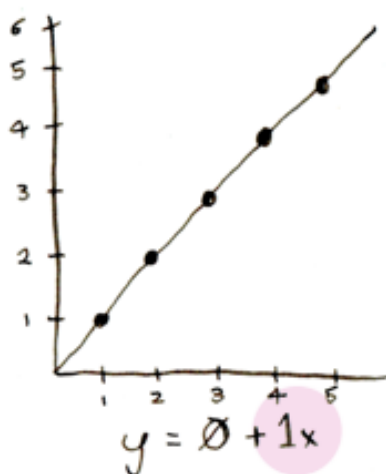
Chúng ta đang vẽ một đường thẳng. Dưới đây là dạng phương trình của một đường thẳng:



Số đầu tiên cho biết đường thẳng bắt đầu ở độ cao bao nhiêu (hay còn gọi là điểm cắt trục tung):

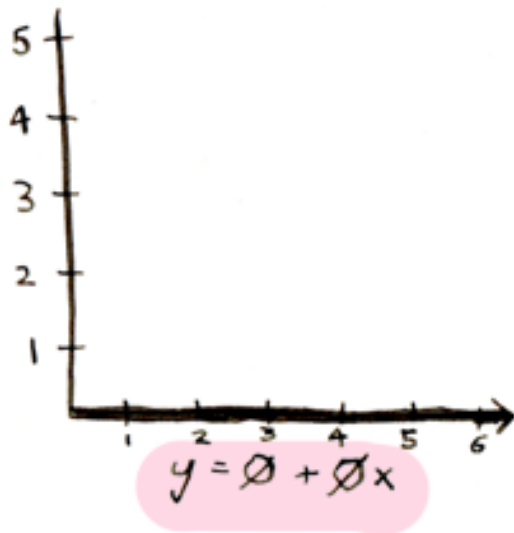


Số thứ 2 cho biết độ nghiêng (góc nghiêng) của đường thẳng



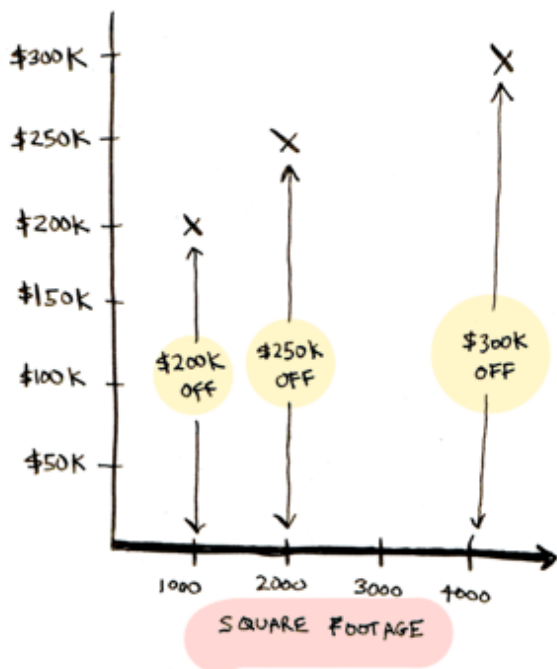
Vì vậy, chúng ta cần tìm ra 2 con số đủ tốt để đường thẳng khớp với dữ liệu. Đây là cách thực hiện:

1. Chọn 2 giá trị khởi đầu. Theo truyền thống, chúng được gọi là θ_0 và θ_1 . Một lựa chọn đầu tiên hợp lý là cả hai bằng 0.
2. Vẽ một đường thẳng sử dụng 2 giá trị đó.



(Đúng vậy, đó là một đường thẳng mà tất cả các giá trị dự đoán đều bằng 0. Dù vậy, đây vẫn là 1 điểm khởi đầu hợp lý – chúng ta sẽ tiếp tục cải thiện nó.)

Bây giờ, hãy tính xem trung bình chúng ta sai lệch bao nhiêu so với dữ liệu thực tế:



Đây được gọi là **hàm chi phí** (cost function). Bạn đưa θ_0 và θ_1 vào hàm này, và nó sẽ cho bạn biết đường thẳng đó sai lệch bao nhiêu (tức là chi phí của việc sử dụng đường thẳng đó).

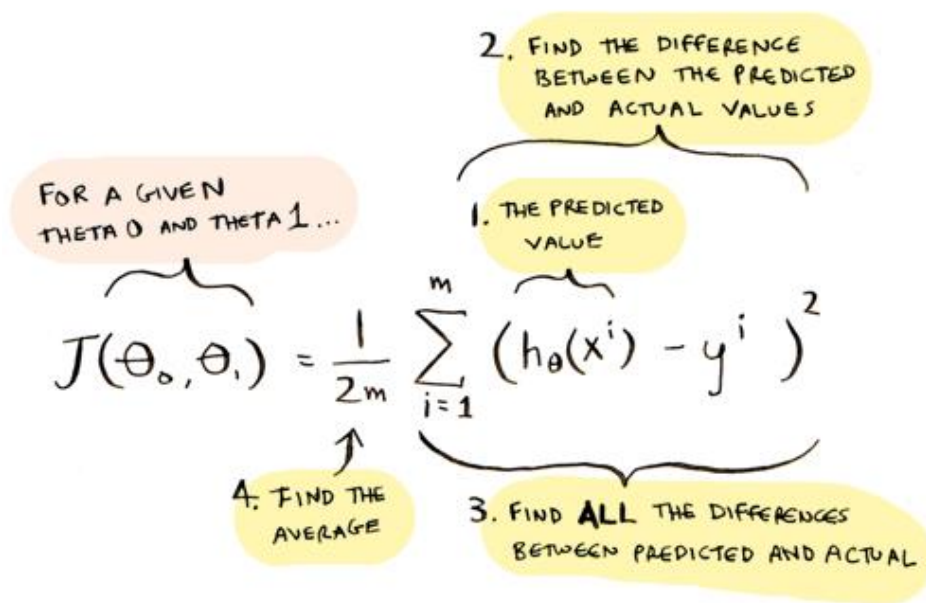
Mình sẽ sử dụng mã Octave, vì đó là ngôn ngữ được dùng trong khóa học.

```

% x is a list of square feet: [1000, 2000, 4000]
% y is the corresponding prices for the homes: [200000, 250000, 300000]
function distance = cost(theta_0, theta_1, x, y)
    distance = 0
    for i = 1:length(x) % arrays in octave are indexed starting at 1
        square_feet = x(i)
        predicted_value = theta_0 + theta_1 * square_feet
        actual_value = y(i)
        % how far off was the predicted value (make sure you get the absolute value)?
        distance = distance + abs(actual_value - predicted_value)
    end
    % get how far off we were on average
    distance = distance / length(x)
end

```

Mình cũng sẽ trình bày công thức toán học mà họ sử dụng cho việc này. Về cơ bản, nó cũng là cùng một ý:



Trong công thức này, $h(x)$ đại diện cho giá trị dự đoán.

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Đó chính là công thức của đường thẳng! Nếu chúng ta thay một giá trị cụ thể của x vào, ta sẽ nhận được giá trị dự đoán $h(x)$.

Vậy nên, $h(x) - y$ cho ta sai lệch giữa giá trị dự đoán và giá trị thực tế.

Với một cặp giá trị θ_0 và θ_1 nhất định, hàm chi phí sẽ cho biết mức độ "tốt" của chúng (tức là cho biết các dự đoán sai lệch bao nhiêu so với dữ liệu thực tế).

Vậy, chúng ta làm gì với thông tin đó? Làm sao để tìm ra giá trị θ_0 và θ_1 tạo nên đường thẳng tốt nhất?

⇒ Bằng cách sử dụng **gradient descent**.

Gradient descent

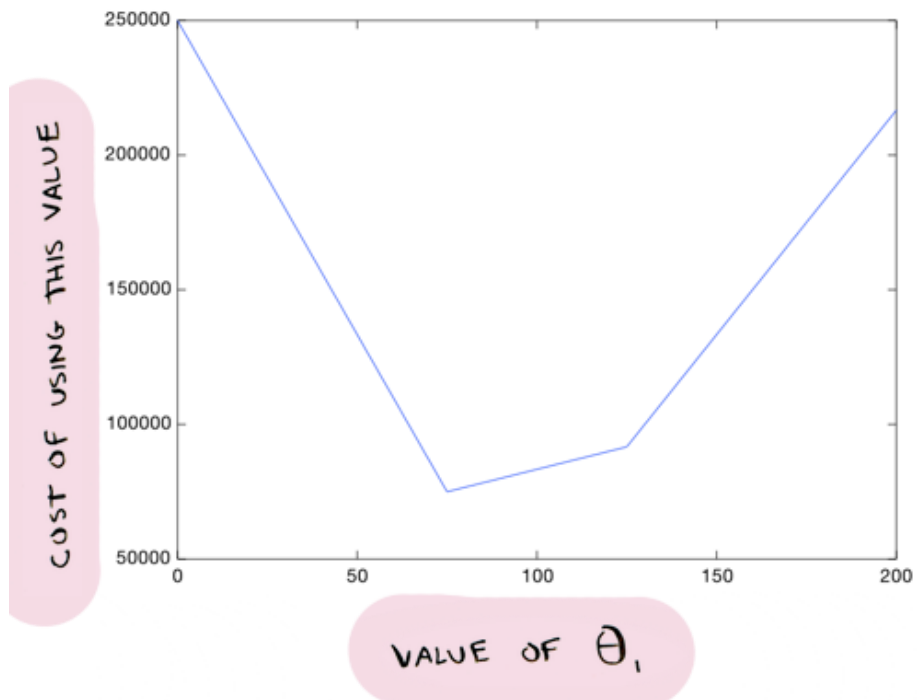


Hãy bắt đầu với một phiên bản đơn giản hơn của **gradient descent**, rồi sau đó chuyển sang phiên bản đầy đủ.

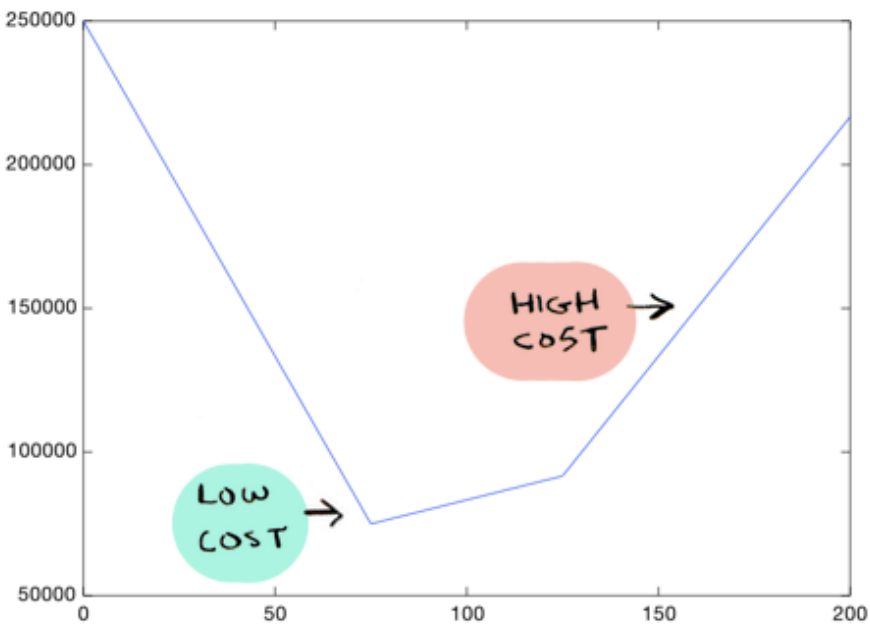
Giả sử ta quyết định giữ $\theta_0 = 0$. Vậy ta chỉ thử nghiệm với các giá trị khác nhau của θ_1 , còn θ_0 luôn bằng 0.

Lúc này, bạn có thể thử nhiều giá trị khác nhau cho θ_1 , và sẽ nhận được các giá trị chi phí khác nhau tương ứng.

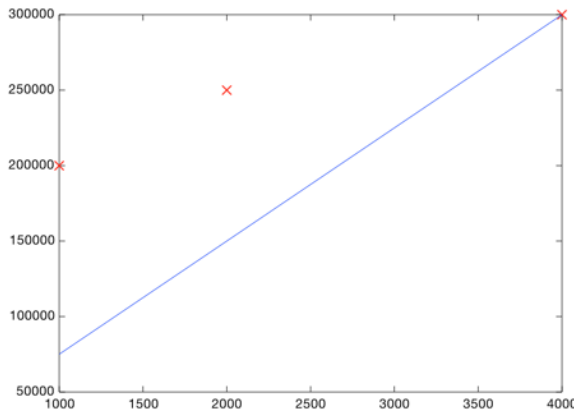
Bạn có thể vẽ tất cả các giá trị chi phí này lên một đồ thị:



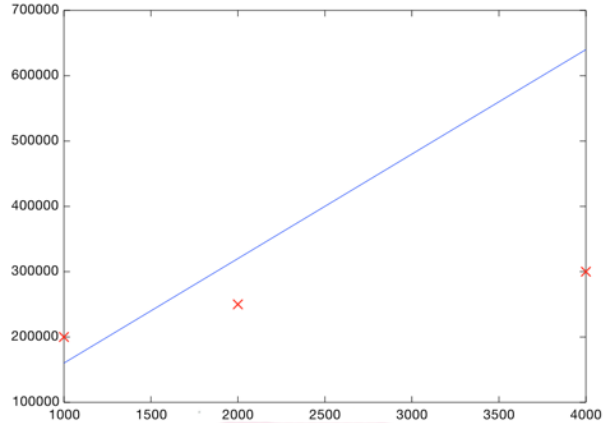
Ví dụ, sử dụng giá trị $\theta_1 = 75$ sẽ tốt hơn so với $\theta_1 = 160$, vì chi phí (mức sai lệch) thấp hơn.



Dưới đây là các đường thẳng tương ứng (nhớ rằng θ_0 bằng 0 trong các đường này):



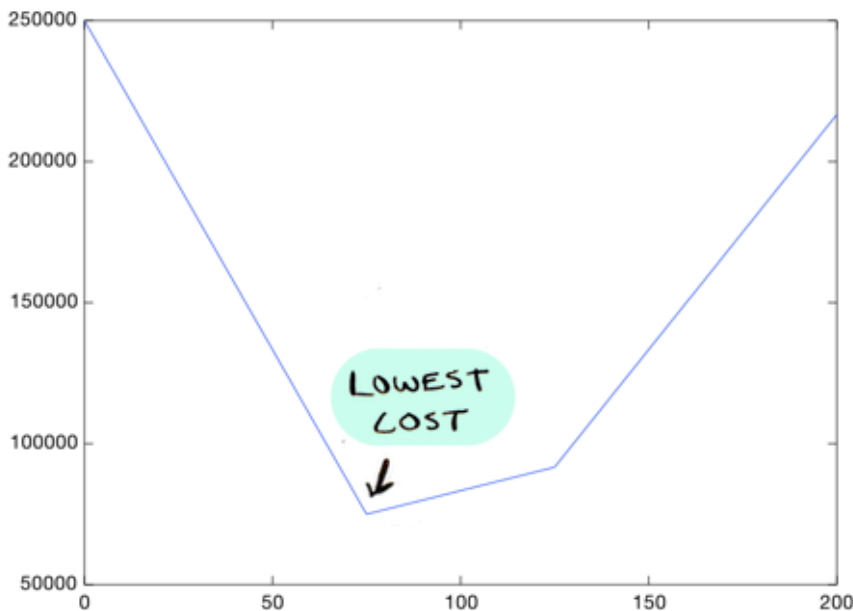
$$\theta_1 = 75$$



$$\theta_1 = 160$$

Ta có thể thấy rằng đường thẳng bên trái khớp với dữ liệu tốt hơn so với đường thẳng bên phải, vì vậy hoàn toàn hợp lý khi chi phí của đường bên trái thấp hơn.

Và từ đồ thị, có vẻ như $\theta_1 = 75$ cho ta chi phí thấp nhất tổng thể.

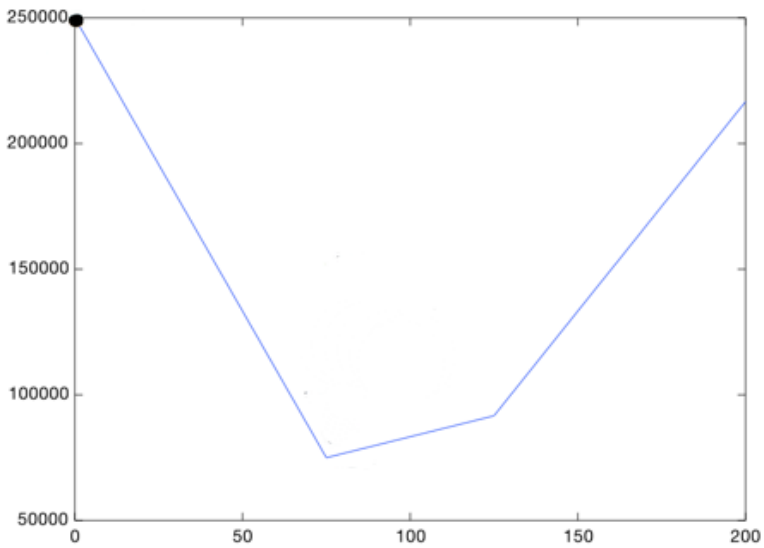


Vì đó là điểm thấp nhất trên đồ thị này. Khi đã vẽ toàn bộ các giá trị chi phí như vậy, ta chỉ cần tìm điểm thấp nhất trên đồ thị - và đó sẽ là giá trị tối ưu của θ_1 .

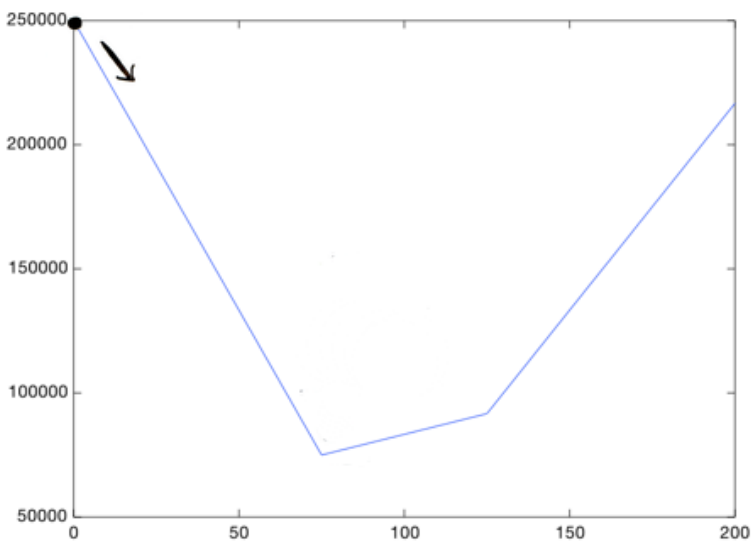
Gradient descent giúp ta tìm điểm thấp nhất trên đồ thị đó. Bạn bắt đầu với một giá trị khởi đầu cho θ_1 , rồi cập nhật dần qua từng bước lặp cho đến khi đạt giá trị tốt nhất.

Ví dụ, bạn có thể bắt đầu với $\theta_1 = 0$.

Sau đó, bạn cần đặt câu hỏi: **nên đi sang trái hay sang phải?**

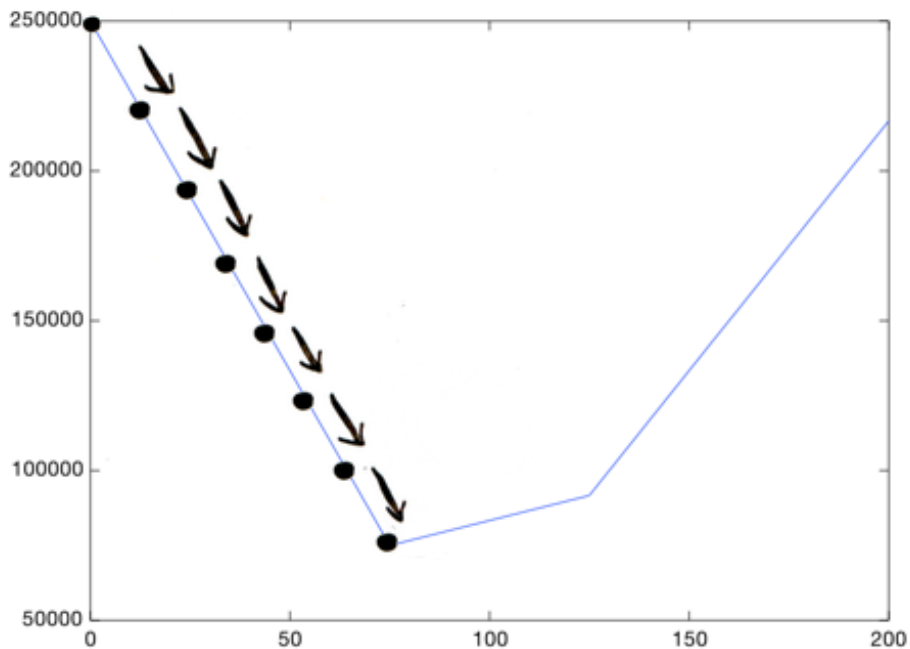


Chúng ta muốn đi xuống (tức là giảm chi phí), vậy hãy thử đi sang phải một bước nhỏ:



Đây là giá trị mới của θ_1 . Một lần nữa, ta lại tự hỏi: **nên đi sang trái hay sang phải?**

Ở mỗi bước, bạn cần tiếp tục đi theo hướng **giảm chi phí**, cho đến khi bạn đến một điểm mà **không thể đi xuống thêm được nữa** – đó chính là điểm thấp nhất, hay giá trị tối ưu của θ_1 .



Đó chính là **gradient descent**: đi xuống từng chút một cho đến khi bạn chạm đáy.

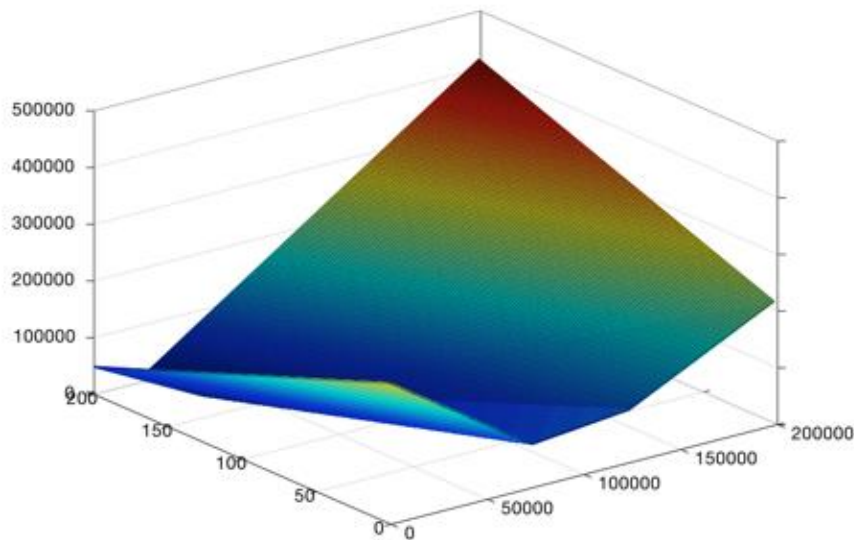
Vậy làm sao biết hướng nào là “xuống”? Câu trả lời có thể rõ ràng với những ai biết vi phân, nhưng không mấy rõ ràng với số còn lại: **bạn lấy đạo hàm tại điểm đó**.

Phần này mình sẽ không đi sâu mà chỉ đưa ra công thức áp dụng. Điều quan trọng cần biết là: nếu bạn lấy giá trị hiện tại của θ_1 và cộng với **đạo hàm tại điểm đó**, bạn sẽ **đi xuống**.

Chỉ cần lặp lại việc này nhiều lần (ví dụ 1000 lần), bạn sẽ đến được điểm thấp nhất!

Đó là phiên bản đơn giản. Giờ hãy quay lại bài toán gốc.

Trong bài toán này, ta cần tìm giá trị tối ưu cho cả θ_0 và θ_1 . Vậy nên đồ thị sẽ giống như thế này:



Bạn có thể thấy rằng nếu $\theta_0 = 200,000$ và $\theta_1 = 200$, thì kết quả sẽ rất tệ. Nhưng nếu $\theta_0 = 100,000$ và $\theta_1 = 50$, thì ổn hơn nhiều.

Đồ thị vẫn là hình dạng cái bát (parabol) với một điểm thấp nhất – chỉ là bây giờ nó là **3D**, vì ta đang xét đến cả θ_0 .

Nhưng ý tưởng thì vẫn giống nhau: **bắt đầu từ một điểm bất kỳ trong “cái bát”, và cứ tiếp tục đi từng bước nhỏ xuống dốc cho đến khi bạn chạm đáy!**

Đây là công thức cho **gradient descent**:

TO UPDATE θ_0 :

ALPHA (α), CONTROLS
HOW FAST θ_0 GETS UPDATED

$$\theta_0 = \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)}_{\text{PARTIAL DERIVATIVE}}$$

TO UPDATE θ_1 :

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i) \cdot x^i$$

Chúng là các đạo hàm riêng được sử dụng trong 2 quy tắc cập nhật: một cho θ_0 và một cho θ_1 .

Hai công thức này gần như giống hệt nhau, chỉ khác là đạo hàm của θ_1 có thêm một thành phần x ở cuối.

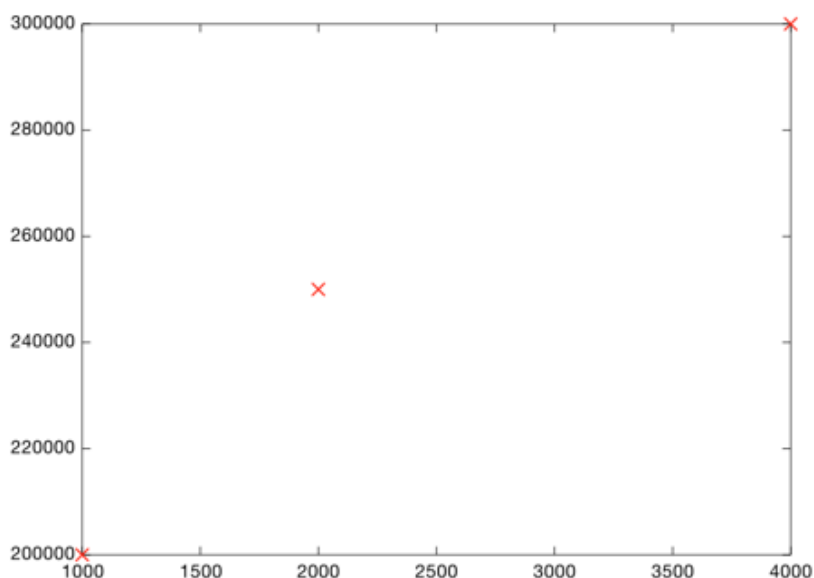
Put in all together

Mã Octave đầy đủ tại đây.

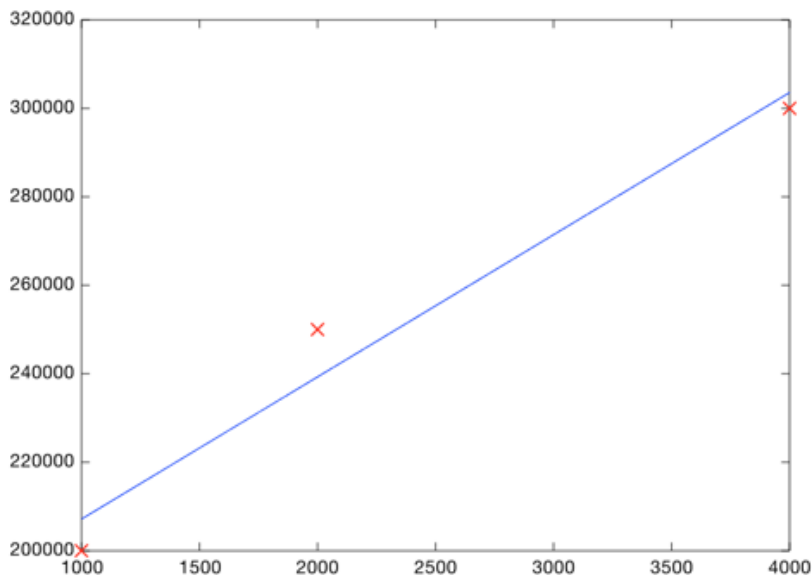
<https://gist.github.com/egonSchiele/2405a8c020e3d2fbdd5a>

Lưu ý: Tôi đã sử dụng một kỹ thuật gọi là **feature scaling** (chuẩn hóa đặc trưng), nên giá trị diện tích được chuyển thành [1, 2, 4] thay vì [1000, 2000, 4000] như ban đầu. Việc này giúp **gradient descent** hoạt động hiệu quả hơn trong ví dụ này.

Chúng ta bắt đầu với các điểm dữ liệu sau:



Sử dụng **gradient descent**, ta tìm được rằng $\theta_0 = 175,000$ và $\theta_1 = 32.143$ cho ra **đường thẳng khớp với dữ liệu tốt nhất**, với **chi phí là \$7,142.9**, và đường thẳng đó trông như sau:



(Đường thẳng ước lượng bằng mắt của chúng ta thực ra rất gần – nó có chi phí là \$8,333.33). Và giờ bạn đã biết rằng bạn có thể bán căn nhà rộng 3,000 feet vuông của mình với giá **271,430**!

Một lưu ý cuối cùng: Octave có sẵn một hàm gọi là `fminunc`, giúp bạn thực hiện phần khó của gradient descent – bạn chỉ cần cung cấp hàm chi phí (cost function).

Đây là ví dụ hàm `fminunc`

<https://gist.github.com/egonSchiele/07a41ebb7654e62f6498>

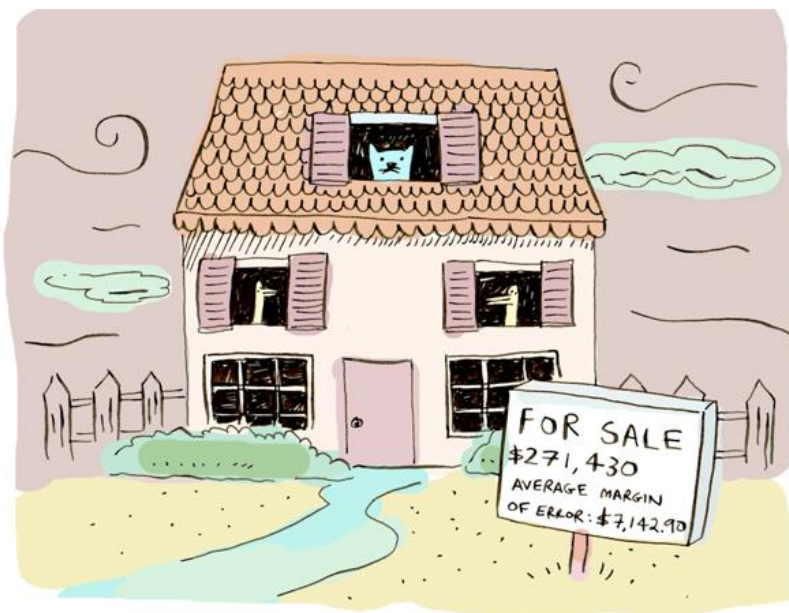
Recap

Hồi quy tuyến tính được sử dụng để **dự đoán một giá trị** (ví dụ: giá bán của một ngôi nhà).

Với một tập dữ liệu cho trước, đầu tiên hãy **vẽ một đường thẳng phù hợp** với nó.

Hàm chi phí cho biết đường thẳng của bạn **phù hợp với dữ liệu tốt đến đâu**.

Bạn có thể sử dụng **gradient descent** để tìm ra **đường thẳng tốt nhất**.



<https://www.adit.io/posts/2016-02-20-Linear-Regression-in-Pictures.html>